MODULE 7

MANISH SHARMA

Python – Collections, functions and Modules

ACCESSING LIST-

Theory:

• UNDERSTANDING HOW TO CREATE AND ACCESS ELEMENTS IN A LIST:-

A **list** in Python is a collection of items stored in a single variable. Lists are used when we need to store multiple values together and access them whenever required. A list can contain different types of data such as numbers, strings, and boolean values.

For example, instead of creating separate variables for the names of students, we can store all the names in a single list.

list_name = [element1, element2, element3]

numbers = [10, 20, 20, 30]

data = ["Manish", 21, 85.5, True]

To use or display a particular element from a list, we access it using its **index number**.

An **index** is the position of an element in a list.

• INDEXING IN LISTS (POSITIVE AND NEGATIVE INDEXING).

Indexing is the process of accessing individual elements of a list using their position number. Every element in a list has a unique position called an **index**.

MODULE 7

MANISH SHARMA

Python provides two types of indexing:

1. **Positive Indexing**
2. **Negative Indexing**

Indexing allows programmers to retrieve, update, or manipulate specific elements in a list.

POSITIVE INDEXING

In Python, positive indexing starts from **0** and moves from left to right.

EXAMPLE LIST

fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]

Element	Index
Apple	0
Banana	1
Mango	2
Orange	3
Grapes	4

NEGATIVE INDEXING

Negative indexing starts from **-1** and moves from right to left.

EXAMPLE LIST

MODULE 7

MANISH SHARMA

fruits = ["Apple", "Banana", "Mango", "Orange", "Grapes"]

Element	Negative Index
Grapes	-1
Orange	-2
Mango	-3
Banana	-4
Apple	-5

• SLICING A LIST: ACCESSING A RANGE OF ELEMENTS.

List Slicing is a technique in Python used to access multiple elements from a list at once. Instead of retrieving one element using indexing, slicing allows us to extract a portion (range) of a list.

Slicing creates a new list containing the selected elements from the original list.

What is Slicing?

Slicing means selecting a specific range of elements from a list.

It is useful when:

- Only a part of the list is needed.
- Data needs to be divided into smaller sections.
- Multiple elements need to be accessed tog

MODULE 7

MANISH SHARMA

TYPES OF LIST SLICING

1. SLICING FROM START TO END

fruits[0:3]

2. OMITTING THE START INDEX

If the start index is omitted, Python starts from index 0.

fruits[:3]

3. OMITTING THE END INDEX

If the end index is omitted, Python goes up to the last element.

fruits[2:]

4. ACCESSING THE ENTIRE LIST

fruits[:]

5. NEGATIVE INDEX SLICING

Negative indexing can also be used in slicing.

fruits[-4:-1]

MODULE 7

MANISH SHARMA

LIST OPERATIONS

• COMMON LIST OPERATIONS: CONCATENATION, REPETITION, MEMBERSHIP.

- **Common List Operations: Concatenation, Repetition, and Membership (Theory)**
- **Introduction**

Python provides several operations that can be performed on lists. These operations help in combining lists, repeating list elements, and checking whether an element exists in a list.

The most common list operations are:

1. **Concatenation**
2. **Repetition**
3. **Membership**

These operations make lists more flexible and useful in programming.

- **1. Concatenation of Lists**
- **Definition**

Concatenation means joining or combining two or more lists into a single list.

Python uses the **+ (plus)** operator for list concatenation.

When lists are concatenated, the elements of the second list are added to the end of the first list, creating a new list.

- **How Concatenation Works**

MODULE 7

MANISH SHARMA

Suppose we have two lists:

- List A contains some elements.
- List B contains some elements.

After concatenation:

- A new list is formed.
- Elements of List A appear first.
- Elements of List B appear after them.

- **Uses of Concatenation**
- Combining student records.
- Merging customer data.
- Joining multiple datasets.
- Creating larger collections from smaller lists.

- **Advantages of Concatenation**
- Easy to combine multiple lists.
- Creates a new list without modifying the originals.
- Useful for organizing and managing data.

- **2. Repetition of Lists**
- **Definition**

Repetition means duplicating the elements of a list multiple times.

MODULE 7

MANISH SHARMA

Python uses the *** (multiplication)** operator for repetition.

When a list is repeated, its elements are copied and added repeatedly according to the specified number.

- **How Repetition Works**

If a list contains several elements and is repeated:

- The entire list is copied.
- Copies are placed one after another.
- The number of copies depends on the repetition value.

For example:

- Repetition by 2 → list appears twice.
- Repetition by 3 → list appears three times.

- **Uses of Repetition**
- Creating repeated patterns.
- Generating sample data.
- Initializing lists with repeated values.
- Building test datasets.

- **Advantages of Repetition**
- Saves time and effort.
- Avoids manually entering repeated values.
- Makes code shorter and cleaner.

MANISH SHARMA

- **3. Membership in Lists**
- **Definition**

Membership is used to check whether a particular element exists in a list.

Python provides two membership operators:

- **1. in**

Checks if an element is present in the list.

- **2. not in**

Checks if an element is not present in the list.

Membership operations return a Boolean value:

- True
- False

- **How Membership Works**

Python searches through the list and compares each element with the specified value.

If the element is found:

- in returns **True**
- not in returns **False**

If the element is not found:

- in returns **False**
- not in returns **True**

MODULE 7

MANISH SHARMA

- **Uses of Membership**
- Searching data.
- Validating user input.
- Checking whether an item exists.
- Preventing duplicate entries.
- Filtering records.

- **Advantages of Membership Operators**
- Easy to use.
- Improves program readability.
- Helps in quick searching.
- Useful in decision-making statements.

• UNDERSTANDING LIST METHODS LIKE APPEND(), INSERT(), REMOVE(), POP().

1. **Understanding List Methods: append(), insert(), remove(), and pop()**
2. **Introduction**

Python provides several built-in methods to modify lists. These methods allow programmers to add new elements, insert elements at specific positions, and remove elements from a list.

Among the most commonly used list methods are:

1. **append()**
2. **insert()**

3. **remove()**
4. **pop()**

These methods help in managing list data efficiently.

---

3. **1. append() Method**
4. **Definition**

The **append()** method is used to add a new element at the **end of a list**.

When append() is used, the new item is placed after the last existing element of the list.

---

5. **Characteristics of append()**

- Adds only one element at a time.
- Inserts the element at the end of the list.
- Modifies the original list.
- Very commonly used for adding data dynamically.

---

6. **Uses of append()**

- Adding new student records.
- Storing user input.
- Building lists dynamically during program execution.
- Maintaining logs or history records.

---

7. **Example**

MANISH SHARMA

Before:

["Apple", "Banana", "Mango"]

After appending "Orange":

["Apple", "Banana", "Mango", "Orange"]

---

8. **2. insert() Method**
9. **Definition**

The **insert()** method is used to add an element at a **specific position (index)** in a list.

Unlike append(), which always adds at the end, insert() allows placing an element anywhere in the list.

---

10. **Characteristics of insert()**

- Requires an index position.
- Existing elements shift to the right.
- Modifies the original list.
- Useful when data must appear at a particular location.

---

11. **Uses of insert()**

- Inserting a student name at a specific roll number position.
- Adding priority tasks in a to-do list.
- Maintaining sorted or organized data.

---

12. **Example**

MANISH SHARMA

Before:

["Apple", "Banana", "Mango"]

Insert "Orange" at index 1:

After:

["Apple", "Orange", "Banana", "Mango"]

13.3. remove() Method

14.Definition

The **remove()** method is used to delete a specific element from a list based on its value.

Python searches for the value and removes its first occurrence.

15.Characteristics of remove()

- Removes an element by value.
- Deletes only the first matching occurrence.
- Modifies the original list.
- Produces an error if the value is not found.

16.Uses of remove()

- Deleting unwanted records.
- Removing completed tasks.
- Eliminating duplicate or unnecessary data.

MANISH SHARMA

17.Example

Before:

["Apple", "Banana", "Mango", "Orange"]

Remove "Banana":

After:

["Apple", "Mango", "Orange"]

18.4. pop() Method

19.Definition

The **pop()** method is used to remove an element using its index position.

If no index is provided, pop() removes the **last element** of the list.

20.Characteristics of pop()

- Removes an element by index.
- Returns the removed element.
- If no index is specified, removes the last item.
- Modifies the original list.

21.Uses of pop()

- Removing the last entered item.
- Managing stacks and queues.
- Deleting elements at specific positions.

22.Example

Before:

["Apple", "Banana", "Mango", "Orange"]

Pop index 2:

After:

["Apple", "Banana", "Orange"]

Removed element:

Mango

WORKING WITH LISTS

- Iterating over a list using loops.

Iteration means accessing each element of a list one by one. In Python, iteration is commonly performed using loops. It allows programmers to process all elements of a list without accessing each element individually.

When working with lists, loops help in:

- Displaying all elements.
- Performing calculations.
- Searching for data.
- Updating values.
- Processing large amounts of information efficiently.

MODULE 7

MANISH SHARMA

WHAT IS ITERATION

Iteration is the process of repeating a set of instructions for every element present in a list.

For example, if a list contains five elements, the loop will execute five times, once for each element.

• SORTING AND REVERSING A LIST USING SORT(), SORTED(), AND REVERSE().

1. **Sorting and Reversing a List Using sort(), sorted(), and reverse()**
2. **Introduction**

When working with lists in Python, it is often necessary to arrange elements in a particular order or reverse their sequence. Python provides built-in methods and functions to perform these operations easily.

The most commonly used tools are:

1. **sort()**
2. **sorted()**
3. **reverse()**

These help in organizing and managing data efficiently.

---

3. **1. Sorting a List Using sort()**
4. **Definition**

The **sort()** method is used to arrange the elements of a list in a specific order.

MANISH SHARMA

By default, sort() arranges elements in **ascending order**.

5. **Characteristics of sort()**

- Modifies the original list.
- Does not create a new list.
- Works only with lists.
- Sorts data in ascending order by default.

6. **Types of Sorting**
7. **Ascending Order**

Smallest to largest or A to Z.

Example:

10, 20, 30, 40, 50

8. **Descending Order**

Largest to smallest or Z to A.

Example:

50, 40, 30, 20, 10

9. **Advantages of sort()**

- Fast and efficient.
- Directly modifies the existing list.
- Useful when the original list needs to be sorted.

10.2. Sorting a List Using sorted()

11. Definition

MODULE 7

MANISH SHARMA

The **sorted()** function is used to sort elements and return a **new sorted list**.

Unlike sort(), it does not change the original list.

12. Characteristics of sorted()

- Creates a new sorted list.
- Keeps the original list unchanged.
- Can be used with lists, tuples, strings, and other iterable objects.
- Returns the sorted result.

13. Advantages of sorted()

- Preserves the original data.
- More flexible than sort().
- Can be used on different data structures.

---

14. Difference Between sort() and sorted()

Feature	sort()	sorted()
Type	List Method	Built-in Function
Original List	Modified	Not Modified
Returns New List	No	Yes
Works On	Lists Only	Any Iterable
Memory Usage	Less	More

MANISH SHARMA

15.3. Reversing a List Using reverse()

16.Definition

The **reverse()** method is used to reverse the order of elements in a list.

It changes the first element to the last position, the second element to the second-last position, and so on.

17.Characteristics of reverse()

- Modifies the original list.
- Does not sort the list.
- Simply changes the order of elements.
- Works only with lists.

18.Example of Reversing

Original Order:

Apple, Banana, Mango, Orange

After Reversing:

Orange, Mango, Banana, Apple

19.Advantages of reverse()

- Simple and fast.
- Useful when data needs to be processed in reverse order.
- Preserves element values while changing their positions.

20.Difference Between Sorting and Reversing

MODULE 7

MANISH SHARMA

Sorting	Reversing
Arranges elements in ascending or descending order	Changes the current order of elements
Uses sort() or sorted()	Uses reverse()
Based on element values	Based on element positions
Produces ordered data	Produces opposite sequence

- BASIC LIST MANIPULATIONS: ADDITION, DELETION, UPDATING, AND SLICING

1 Basic List Manipulations (Addition, Deletion, Updating, Slicing)

2 Given List

students = ["priya", "vivek", "aditya", "gopal", "keyur", "manish"]

---

3 1. Addition (Adding Elements)

4 Add new names:

students.append("rahul")

students.append("neha")

5 Updated List:

["priya", "vivek", "aditya", "gopal", "keyur", "manish", "rahul", "neha"]

---

6 2. Deletion (Removing Elements)

MANISH SHARMA

7 Remove "aditya":

students.remove("aditya")

8 Updated List:

["priya", "vivek", "gopal", "keyur", "manish", "rahul", "neha"]

---

9 3. Updating (Changing Elements)

10 Change "gopal" to "arjun":

students[2] = "arjun"

11 Updated List:

["priya", "vivek", "arjun", "keyur", "manish", "rahul", "neha"]

---

12 4. Slicing (Accessing Range of Elements)

13 Get first 3 students:

students[0:3]

14 Output:

["priya", "vivek", "arjun"]

---

15 Get last 2 students:

students[-2:]

16 Output:

["rahul", "neha"]

MODULE 7

MANISH SHARMA

17 Final Summary List

After all operations, the final list is:

["priya", "vivek", "arjun", "keyur", "manish", "rahul", "neha"]

---

18 Key Points

- **Addition** → append() adds new elements.
- **Deletion** → remove() deletes specific elements.
- **Updating** → change value using index.
- **Slicing** → access a range using [start:end].

4 TUPLE

4.1• INTRODUCTION TO TUPLES, IMMUTABILITY

Immutability means that once a tuple is created, its elements **cannot be changed, added, or removed**.

In simple words:
👉 A tuple is **unchangeable** after creation.

---

19 Example of Immutability

```
my_tuple = ("apple", "banana", "mango")
my_tuple[1] = "grapes"   # This will give an error
```

4.2• CREATING AND ACCESSING ELEMENTS IN A TUPLE

MODULE 7

MANISH SHARMA

20 Creating and Accessing Elements in a Tuple

21 Introduction

A **tuple** is a collection in Python used to store multiple values in a single variable. Tuples are similar to lists, but the main difference is that tuples are **immutable**, meaning their elements cannot be changed after creation.

---

22 Creating a Tuple

23 Definition

A tuple is created by placing elements inside **round brackets ( )**, separated by commas.

---

24 Syntax

tuple_name = (element1, element2, element3)

---

25 Example

fruits = ("apple", "banana", "mango", "orange")

Here:

- fruits is a tuple
- It contains 4 elements

---

26 Tuple with Different Data Types

data = ("Manish", 21, 85.5, True)

A tuple can store:

MODULE 7

MANISH SHARMA

- Strings
- Integers
- Floats
- Booleans

---

27 Empty Tuple

empty_tuple = ()

---

28 Single Element Tuple

For a single element, a comma is required:

t1 = ("apple",)

Without comma, Python will not treat it as a tuple.

---

29 Accessing Elements in a Tuple

30 Definition

Tuple elements are accessed using **indexing**, just like lists.

Indexing starts from **0**.

---

31 Positive Indexing

32 Example

fruits = ("apple", "banana", "mango", "orange")

MANISH SHARMA

Element	Index
apple	0
banana	1
mango	2
orange	3

---

33 Access Example

print(fruits[1])

Output:

banana

---

34 Negative Indexing

Negative indexing starts from **-1** (from the end).

Element	Negative Index
orange	-1
mango	-2
banana	-3
apple	-4

MANISH SHARMA

35 Access Example

print(fruits[-1])

Output:

orange

---

36 Accessing Multiple Elements (Slicing)

Tuples also support **slicing**.

37 Syntax

tuple_name[start:end]

- Start index is included
- End index is excluded

---

38 Example

fruits = ("apple", "banana", "mango", "orange", "grapes")

print(fruits[1:4])

Output:

('banana', 'mango', 'orange')

---

39 Key Points to Remember

- Tuples are created using **()**
- Tuples are **ordered and indexed**
- Indexing starts from **0**

- Negative indexing starts from **-1**
- Tuples are **immutable**
- Elements can be accessed but cannot be changed
- Slicing is used to access a range of elements

4.3• BASIC OPERATIONS WITH TUPLES: CONCATENATION, REPETITION, MEMBERSHIP.

40 Basic Operations with Tuples: Concatenation, Repetition, Membership

41 Introduction

A **tuple** is a collection in Python used to store multiple values in a single variable. Tuples are **ordered and immutable**, meaning their elements cannot be changed after creation.

Even though tuples are immutable, we can still perform several operations on them such as:

1. Concatenation
2. Repetition
3. Membership

---

42 1. Tuple Concatenation

43 Definition

Concatenation means joining two or more tuples into a single tuple.

In Python, the **+ operator** is used for concatenation.

MANISH SHARMA

44 How It Works

- Two tuples are combined.
- A new tuple is created.
- Original tuples remain unchanged.

---

45 Example

t1 = ("apple", "banana")

t2 = ("mango", "orange")

result = t1 + t2

print(result)

46 Output:

('apple', 'banana', 'mango', 'orange')

---

47 Advantages of Concatenation

- Easy to combine data.
- Does not modify original tuples.
- Useful for merging datasets.

---

48 2. Tuple Repetition

49 Definition

Repetition means repeating the elements of a tuple multiple times.

MANISH SHARMA

The * **operator** is used for repetition.

5. ACCESSING TUPLES

5.1• Accessing tuple elements using positive and negative indexing.

Introduction

A **tuple** in Python is an ordered collection of elements that is **immutable** (cannot be changed after creation). Even though tuples cannot be modified, their elements can still be accessed using **indexing**.

Indexing is used to retrieve specific elements from a tuple based on their position.

Python supports two types of indexing:

1. **Positive Indexing**
2. **Negative Indexing**

---

50 1. Positive Indexing in Tuples

51 Definition

Positive indexing means accessing elements from the beginning of the tuple. It starts from index **0**.

---

52 Example Tuple

fruits = ("apple", "banana", "mango", "orange", "grapes")

---

53 Index Table

MODULE 7

MANISH SHARMA

Element	Index
apple	0
banana	1
mango	2
orange	3
grapes	4

---

54 Accessing Elements

55 Example

```
print(fruits[0])
print(fruits[2])
```

56 Output

```
apple
mango
```

---

57 Key Points (Positive Indexing)

- Starts from **0**
- Moves left to right
- Used when position

MODULE 7

MANISH SHARMA

5.2• SLICING A TUPLE TO ACCESS RANGES OF ELEMENTS.

Tuple slicing means getting a part (subset) of a tuple using indexes.

58 Syntax:

tuple[start : end : step]

- **start** → where slicing begins (included)
- **end** → where slicing stops (not included)
- **step** → jump size (optional)

---

59 Example Tuple:

t = (10, 20, 30, 40, 50, 60, 70)

---

60 🔹 1. Basic Slicing

print(t[1:4])

Output:

(20, 30, 40)

Starts at index 1 and goes till 3 (4 is excluded)

---

61 🔹 2. From Beginning

print(t[:3])

Output:

(10, 20, 30)

---

62 🔹 3. Till End

print(t[3:])

Output:

(40, 50, 60, 70)

---

63 🔹 4. With Step

print(t[0:7:2])

Output:

(10, 30, 50, 70)

---

64 🔹 5. Negative Index Slicing

print(t[-5:-1])

Output:

(30, 40, 50, 60)

6.DICTIONARIES

6.1• INTRODUCTION TO DICTIONARIES: KEY-VALUE PAIRS.

A **dictionary** in Python is a collection that stores data in **key-value pairs**.

---

65 What is a Key–Value Pair?

Each item in a dictionary has:

- **Key** → unique identifier (like a name or ID)

- **Value** → data related to that key

Example:

"name": "Manish"

Here:

- "name" = key
- "Manish" = value

---

66 Syntax of Dictionary:

```
dict_name = {
   key1: value1,
   key2: value2,
   key3: value3
}
```

---

67 Example:

```
student = {
   "name": "kajal",
   "age": 26,
   "course": "Python"
}


print(student)
```

Output:

{'name': 'kajal', 'age': 26, 'course': 'Python'}

---

68 🔹 Accessing Values:

print(student["name"])

Output:

Priya

---

69 🔹 Characteristics of Dictionaries:

- Keys must be **unique**
- Values can be **any data type**
- Unordered (before Python 3.7, but now insertion order is preserved)
- Mutable (can be changed)

---

70 🔹 Example with Different Data Types:

```
data = {
  "id": 101,
  "marks": 89.5,
  "passed": True
```

6.2• ACCESSING, ADDING, UPDATING, AND DELETING DICTIONARY ELEMENTS.

MODULE 7

MANISH SHARMA

A **dictionary** in Python is a collection data type that stores data in **key-value pairs**. Each key is unique and is used to access its corresponding value. Dictionaries are **mutable**, meaning their elements can be changed after creation.

---

71 🔹 1. Accessing Elements

Accessing means retrieving the value stored in a dictionary using its key.

- Each value is accessed using the **key inside square brackets** or using the get() method.
- If the key does not exist, square bracket access gives an error, but get() returns None.

Example idea:
To get a student's name from a record, we use the key "name".

---

72 🔹 2. Adding Elements

Adding means inserting a new key-value pair into the dictionary.

- A new key is created and assigned a value.
- If the key already exists, it will not add a new entry but will be updated (overwritten).

Example idea:
Adding "city": "Ahmedabad" to a student record.

---

73 🔹 3. Updating Elements

Updating means changing the value of an existing key.

- If the key exists, its value is replaced with a new value.
- If the key does not exist, it behaves like adding a new key-value pair.

Example idea:
Changing "age": 20 to "age": 21.

---

74 🔹 4. Deleting Elements

Deleting means removing a key-value pair from the dictionary.

- This can be done using:
    - del keyword
    - pop() method
- del directly removes the key-value pair.
- pop() removes the key and also returns its value.

Example idea:
Removing "course" from a student record.

6.2• DICTIONARY METHODS LIKE KEYS(), VALUES(), AND ITEMS().

MODULE 7

MANISH SHARMA

75 DICTIONARY METHODS IN PYTHON: KEYS(), VALUES(), AND ITEMS() (THEORY)

PYTHON DICTIONARIES PROVIDE BUILT-IN METHODS TO ACCESS DIFFERENT PARTS OF THE DATA STORED IN KEY-VALUE PAIRS. THE MOST COMMONLY USED METHODS ARE KEYS(), VALUES(), AND ITEMS().

---

76 ◇ 1. KEYS() METHOD

THE KEYS() METHOD IS USED TO RETRIEVE ALL THE KEYS FROM A DICTIONARY.

- IT RETURNS A VIEW OBJECT THAT DISPLAYS ALL KEYS.
- IT IS USEFUL WHEN WE WANT TO WORK ONLY WITH DICTIONARY KEYS.

👉 IN SIMPLE TERMS, IT GIVES ALL THE “LABELS” OF THE DATA.

---

77 ◇ 2. VALUES() METHOD

THE VALUES() METHOD IS USED TO RETRIEVE ALL THE VALUES FROM A DICTIONARY.

- IT RETURNS A VIEW OBJECT CONTAINING ONLY VALUES.
- KEYS ARE NOT INCLUDED IN THE RESULT.

IN SIMPLE TERMS, IT GIVES ALL THE “DATA” STORED IN THE DICTIONARY.

78 ◇ 3. ITEMS() METHOD

THE ITEMS() METHOD IS USED TO RETRIEVE ALL KEY-VALUE PAIRS FROM A DICTIONARY.

- IT RETURNS EACH ELEMENT AS A TUPLE (KEY, VALUE).
- IT IS VERY USEFUL WHEN BOTH KEYS AND VALUES ARE NEEDED TOGETHER.

IN SIMPLE TERMS, IT GIVES COMPLETE RECORDS.

---

79 IMPORTANT POINTS:

- ALL THREE METHODS RETURN VIEW OBJECTS, NOT LISTS.
- THEY AUTOMATICALLY REFLECT ANY CHANGES MADE IN THE DICTIONARY.

MODULE 7

- **THEY ARE COMMONLY USED IN LOOPS AND DATA PROCESSING.**

---

80 REAL-LIFE ANALOGY:

THINK OF A STUDENT RECORD SYSTEM:

- **KEYS() → FIELD NAMES LIKE NAME, AGE, COURSE**
- **VALUES() → ACTUAL DATA LIKE** KAJAL ,26, **PYTHON**
- **ITEMS() → FULL RECORDS LIKE (NAME:** KAJAL**)**

7. WORKING WITH DICTIONARIES

7.1• ITERATING OVER A DICTIONARY USING LOOPS

81 📘 Iterating Over a Dictionary Using Loops (Theory)

Iterating over a dictionary means **accessing each element of the dictionary one by one using a loop**, usually a for loop. This helps in reading and processing all keys, values, or both key-value pairs in an organized way.

---

82 🔹 Purpose of Iteration

Iteration is used to:

- Access all keys and values in a dictionary
- Perform operations on each item
- Display or process data step by step

---

83 🔹 Iterating Over Keys

By default, when we loop through a dictionary, it returns **only keys**.

- It helps in accessing all the identifiers in the dictionary.
- Useful when we only need keys.

---

84 🔹 Iterating Using keys() Method

The keys() method is used to explicitly loop through all keys.

- It provides better readability in code.
- Functionally similar to default iteration.

---

85 🔹 Iterating Using values() Method

The values() method is used when we want to access only the **values** of the dictionary.

- It ignores keys and focuses only on stored data.

---

86 🔹 Iterating Using items() Method

The items() method is used to access both **keys and values together**.

- Each element is returned as a tuple (key, value).
- It is the most commonly used method for full dictionary traversal.

• MERGING TWO LISTS INTO A DICTIONARY USING LOOPS OR ZIP().

87 📘 Merging Two Lists into a Dictionary Using Loops or zip() (Theory)

MODULE 7

MANISH SHARMA

In Python, we can combine two lists to create a **dictionary** by using either a **loop** or the built-in function zip(). This process is useful when we want to pair related data together in key-value form.

---

88 🔹 What Does Merging Mean?

Merging means combining:

- One list as **keys**
- Another list as **values**

The result is a **dictionary with key-value pairs**.

---

89 🔹 1. Using Loops

In this method, we manually pair elements from both lists using indexing.

- We use a for loop and range().
- Each element from the first list becomes a key.
- Each element from the second list becomes a value.

This method gives more control over the process.

---

90 🔹 2. Using zip() Function

The zip() function automatically pairs elements from two lists.

- It combines elements based on their position.
- It is shorter and more efficient than loops.

It creates pairs like: (key, value).

MODULE 7

MANISH SHARMA

• COUNTING OCCURRENCES OF CHARACTERS IN A STRING USING DICTIONARIES.

91 📘 Counting Occurrences of Characters in a String Using Dictionaries (Theory)

In Python, a dictionary can be used to **count how many times each character appears in a string**. This is a very common problem in data processing and text analysis.

---

92 🔹 What is Character Counting?

Character counting means finding:

- How many times each letter, digit, or symbol appears in a string.

Example:
String = "apple"

- a → 1
- p → 2
- l → 1
- e → 1

---

93 🔹 How Dictionaries Help?

A dictionary is ideal because:

- **Key → character**
- **Value → frequency (count)**

So each character is stored as a key, and its occurrence count is stored as the value.

---

94 🔹 Working Process:

1. Start with an empty dictionary
2. Read each character in the string one by one
3. If character is already in dictionary → increase its count
4. If not → add it with value 1

---

95 🔹 Using Loop Logic:

- Iterate through the string using a loop
- Check each character
- Update dictionary accordingly

8. FUNCTIONS

8.**1 WHAT IS A FUNCTION IN PYTHON?**

A function in Python is a **block of reusable code** that performs a specific task. It helps in organizing programs, reducing repetition, and improving readability. A function runs only when it is called.

8.2• DIFFERENT TYPES OF FUNCTIONS: WITH/WITHOUT PARAMETERS, WITH/WITHOUT RETURN VALUES

MODULE 7

MANISH SHARMA

Type of Function	Meaning	Key Features	Example Use
Function without parameters	Function that does not take any input values	- No arguments passed- Works on fixed data inside function	Displaying a message like “Hello World”
Function with parameters	Function that takes input values while calling	- Accepts arguments- More flexible and reusable	Adding two numbers given by user
Function without return value	Function that performs task but does not return result	- Uses print()- Cannot store output in variable	Printing result directly
Function with return value	Function that returns output using return keyword	- Output can be stored- Useful for calculations	Returning sum of two numbers

8.3. • ANONYMOUS FUNCTIONS (LAMBDA FUNCTIONS).

Anonymous functions in Python are also called **lambda functions**. These are small, single-line functions that do not have a name. They are used to write simple and short functions in a compact form.

A lambda function is defined using the **lambda keyword** instead of the def keyword. It can take any number of arguments but contains only **one expression**. The result of the expression is automatically returned without using the return keyword.

MODULE 7

MANISH SHARMA

Lambda functions are mainly used when a function is required for a short period of time and does not need a formal definition. They are often used with functions like map(), filter(), and sorted() for quick processing of data.

9. MODULES

9.1• INTRODUCTION TO PYTHON MODULES AND IMPORTING MODULES

96 Introduction to Python Modules and Importing Modules (Theory)

A **module** in Python is a file that contains Python code such as functions, variables, and classes. Modules help in **organizing code into separate files**, making programs easier to manage, reuse, and maintain.

Python provides many built-in modules, and we can also create our own modules.

---

97 🔹 What is a Module?

A module is simply a **Python file (.py)** that contains reusable code.

Example:

- A file named math_operations.py can be a module if it contains functions like add, subtract, etc.

---

98 🔹 Why Modules are Used?

Modules are used to:

- Organize code into separate files
- Reuse code in multiple programs
- Improve readability and maintenance

MODULE 7

MANISH SHARMA

- Avoid repetition of code

---

99 🔹 Importing Modules

To use a module in Python, we need to **import** it using the import keyword.

Once imported, we can use its functions and variables in our program.

---

100 🔹 Types of Importing Modules

1011. Import Entire Module

Used to import the full module.

1022. Import Specific Function or Variable

Used when we need only selected parts of a module.

1033. Import with Alias

Used to give a short name to a module for easier use.

9.2• STANDARD LIBRARY MODULES: MATH, RANDOM.

104 Standard Library Modules in Python: math and random (Theory)

Python provides a large collection of **built-in modules** known as the **standard library**. These modules contain ready-made functions that help in performing common tasks without writing extra code. Two important standard library modules are **math** and **random**.

MODULE 7

MANISH SHARMA

105 🔹 1. math Module

The math module provides **mathematical functions and constants** used in calculations.

It is used for operations like:

- Square root
- Power
- Rounding numbers
- Trigonometric calculations

It helps in solving mathematical problems easily and accurately.

---

106 Key Features of math Module:

- Contains advanced mathematical functions
- Provides constants like π (pi) and e
- Used in scientific and engineering calculations

---

107 🔹 2. random Module

The random module is used to generate **random numbers or random selections**.

It is commonly used for:

- Generating random integers
- Selecting random values from a list
- Creating randomness in games and simulations

9.3• CREATING CUSTOM MODULES.

MODULE 7

MANISH SHARMA

A **custom module** in Python is a user-defined file that contains Python code such as functions, variables, or classes. It is created by the programmer to organize code in a structured way and reuse it in different programs.

To create a custom module, we write Python code in a file with a **.py extension** and save it with a meaningful name. This file acts as a module. We can then use this module in other Python programs by importing it using the **import keyword**.

Once a module is imported, all the functions and variables defined inside it can be accessed and used in the main program. This helps in avoiding repetition of code and improves code reusability.

Custom modules make programs easier to manage because large programs can be divided into smaller parts. Each part can be written and tested separately, which improves readability and maintenance.